

Experiment – 7

Mean, Standard Deviation, and Correlation Coefficient

AIM: To compute the Mean, Standard Deviation, and Correlation Coefficient of a given image.

Theory

1. Mean (m)

The mean intensity of an image represents the **average brightness** of the image. It is computed as:

$$m = (1/MN) \sum_{x=1}^M \sum_{y=1}^N I(x, y)$$

where:

- M and N are the dimensions of the image
- I(x,y) is the intensity value at pixel (x,y)

2. Standard Deviation (σ)

The standard deviation measures the contrast or spread of pixel intensities in an image.

$$\sigma = \sqrt{(1/MN) \sum_{x=1}^M \sum_{y=1}^N (I(x, y) - m)^2}$$

A higher standard deviation indicates higher contrast.

3. Correlation Coefficient (r)

The correlation coefficient measures the degree of similarity between two images or between an image and its shifted version.

$$r = \sum (A_i - \bar{A})(B_i - \bar{B}) / \sqrt{\sum (A_i - \bar{A})^2 \sum (B_i - \bar{B})^2}$$

Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_color = cv2.imread('./content/tiger.jpg')
img_color = cv2.cvtColor(img_color, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img_color, cv2.COLOR_RGB2GRAY)

cropped_img = img_gray[0:200, 0:200]
mean_val = np.mean(cropped_img)
std_val = np.std(cropped_img)
size = 200
```

```

block = 20

image1 = np.zeros((size, size))
for i in range(0, size, block):
    for j in range(0, size, block):
        if (i//block + j//block) % 2 == 0:
            image1[i:i+block, j:j+block] = 255

block2 = 10
image2 = np.zeros((size, size))
for i in range(0, size, block2):
    for j in range(0, size, block2):
        if (i//block2 + j//block2) % 2 == 0:
            image2[i:i+block2, j:j+block2] = 255

corr = np.corrcoef(image1.flatten(), image2.flatten())[0,1]
plt.figure(figsize=(10,10))
# Original Image
plt.subplot(3,2,1)
plt.imshow(img_color)
plt.title("Original Image")
plt.axis('on')

# Gray Image
plt.subplot(3,2,2)
plt.imshow(img_gray, cmap='gray')
plt.title("Gray Image")
plt.axis('on')

# Cropped Image
plt.subplot(3,2,3)
plt.imshow(cropped_img, cmap='gray')
plt.title("Cropped Image")
plt.axis('on')

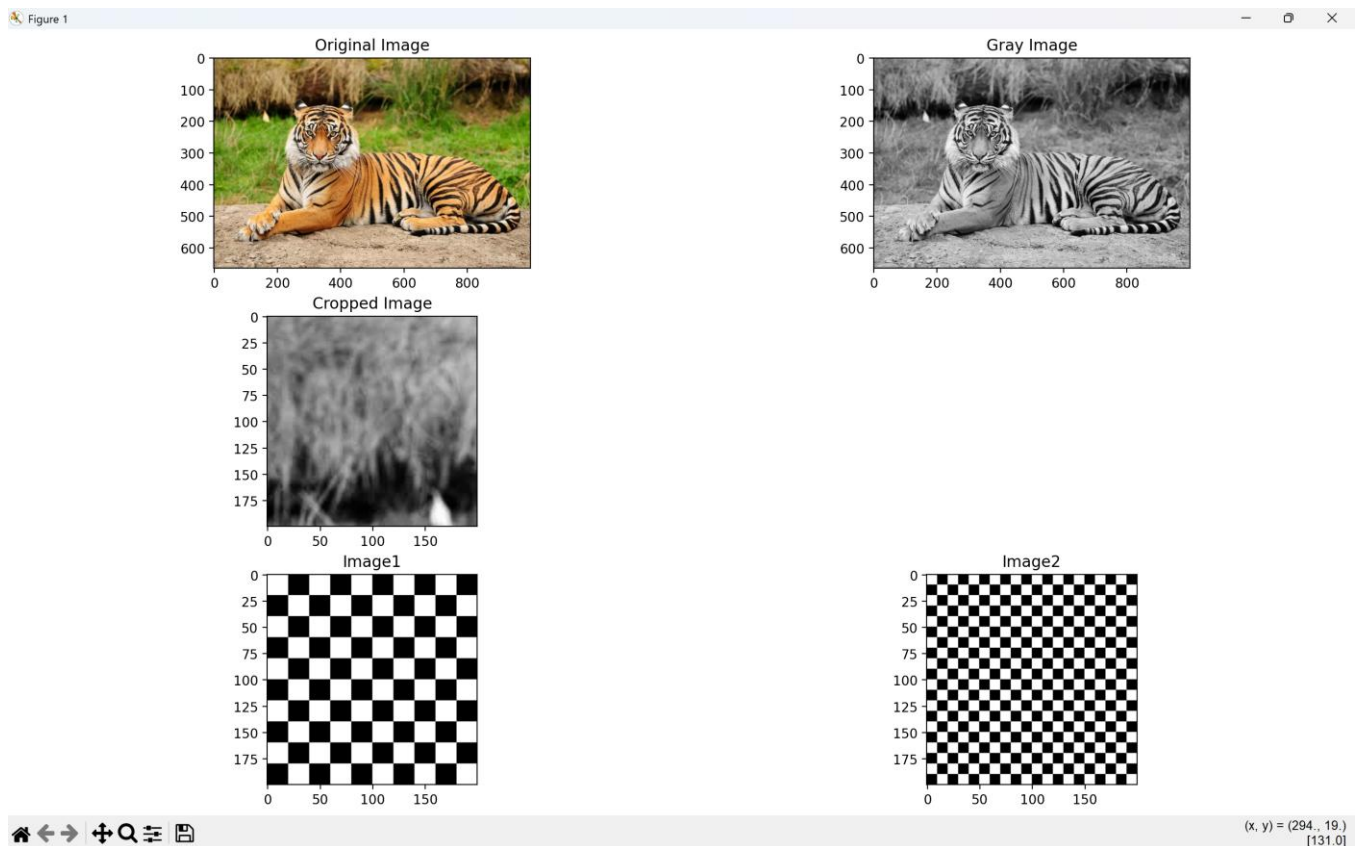
# Image1
plt.subplot(3,2,5)
plt.imshow(image1, cmap='gray')
plt.title("Image1")
plt.axis('on')

# Image2
plt.subplot(3,2,6)
plt.imshow(image2, cmap='gray')
plt.title("Image2")
plt.axis('on')
plt.tight_layout()
plt.show()

# Print Mean, Std, Correlation
print("Mean (m):", round(mean_val,4))
print("Standard Deviation (s):", round(std_val,4))
print("Correlation Coefficient (r):", round(corr,4))

```

Output



```
Mean (m): 104.4932
Standard Deviation (s): 43.7772
Correlation Coefficient (r): 0.0
```

Results and Discussion

The RGB image was loaded and converted to grayscale, and a 200×200 cropped region was analyzed. The cropped image had a mean intensity of **104.4932** and a standard deviation of **43.7772**. Two checkerboard images with different block sizes were also generated, and their correlation coefficient was calculated.

The mean value indicates the region is fairly bright, while the relatively low standard deviation suggests low contrast and smooth intensity variation. The correlation coefficient helps measure similarity: values near 1 indicate strong similarity, near 0 indicate no relation, and near -1 indicate negative correlation.

Experiment – 9

Image Sharpening and Edge Detection

AIM: To implement image sharpening and edge detection on a grayscale image using Laplacian and gradient-based filters (Sobel, Prewitt, and Roberts).

Theory

Image sharpening and edge detection are important techniques in digital image processing used to highlight boundaries and fine details in an image.

The Laplacian filter is a second-order derivative operator that detects regions of rapid intensity change. It enhances edges by emphasizing high-frequency components. Image sharpening can be achieved by subtracting the Laplacian from the original image.

Sobel, Prewitt, and Roberts are gradient-based edge detection operators that compute the first-order derivatives of the image intensity.

- The Sobel operator uses weighted kernels to compute horizontal and vertical gradients and provides better noise resistance.
- The Prewitt operator is similar to Sobel but uses simpler kernels.
- The Roberts operator uses small 2×2 kernels and detects edges along diagonal directions but is more sensitive to noise. These operators help detect edges by calculating the gradient magnitude of the image.

Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('./content/pokemon.jpeg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float64)

lap = cv2.Laplacian(gray.astype(np.uint8), cv2.CV_64F, ksize=3)
sharp = np.clip(gray - lap, 0, 255).astype(np.uint8)

sx = cv2.Sobel(gray.astype(np.uint8), cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(gray.astype(np.uint8), cv2.CV_64F, 0, 1, ksize=3)
edge_sobel = np.uint8(np.clip(np.sqrt(sx**2 + sy**2), 0, 255))

kx = np.array([[ -1, 0, 1], [ -1, 0, 1], [ -1, 0, 1]], dtype=np.float64)
ky = np.array([[ -1, -1, -1], [ 0, 0, 0], [ 1, 1, 1]], dtype=np.float64)
gx = cv2.filter2D(gray.astype(np.uint8), cv2.CV_64F, kx)
gy = cv2.filter2D(gray.astype(np.uint8), cv2.CV_64F, ky)
edge_prewitt = np.uint8(np.clip(np.sqrt(gx**2 + gy**2), 0, 255))

kx = np.array([[ 0, 1], [ -1, 0]], dtype=np.float64)
ky = np.array([[ 1, 0], [ 0, -1]], dtype=np.float64)
gx = cv2.filter2D(gray.astype(np.uint8), cv2.CV_64F, kx)
```

```

gy = cv2.filter2D(gray.astype(np.uint8), cv2.CV_64F, ky)
edge_roberts = np.uint8(np.clip(np.sqrt(gx**2 + gy**2), 0, 255))

lap_edges = np.uint8(np.clip(np.abs(lap), 0, 255))
fig, ax = plt.subplots(2, 3, figsize=(12, 8))
ax[0, 0].imshow(gray.astype(np.uint8), cmap='gray')
ax[0, 0].set_title('Original')
ax[0, 0].axis('off')
ax[0, 1].imshow(sharp, cmap='gray')
ax[0, 1].set_title('Sharpened (Laplacian)')
ax[0, 1].axis('off')
ax[0, 2].imshow(lap_edges, cmap='gray')
ax[0, 2].set_title('Laplacian Edges')
ax[0, 2].axis('off')
ax[1, 0].imshow(edge_sobel, cmap='gray')
ax[1, 0].set_title('Sobel')
ax[1, 0].axis('off')
ax[1, 1].imshow(edge_prewitt, cmap='gray')
ax[1, 1].set_title('Prewitt')
ax[1, 1].axis('off')
ax[1, 2].imshow(edge_roberts, cmap='gray')
ax[1, 2].set_title('Roberts')
ax[1, 2].axis('off')
plt.tight_layout()
plt.show()

```

Output



Results and Discussion

The program successfully implemented image sharpening using the Laplacian filter and performed edge detection using Sobel, Prewitt, and Roberts operators. The sharpened image shows enhanced fine details and improved clarity compared to the original image. The Laplacian edge image clearly highlights regions of rapid intensity change. Among the gradient-based operators, the Sobel filter produced smoother and more defined edges due to its weighted kernel, while the Prewitt filter gave similar but slightly less pronounced results. The Roberts operator detected edges effectively but appeared more sensitive to noise due to its smaller kernel size. Overall, the experiment demonstrates that Laplacian is effective for sharpening, while Sobel provides better edge detection performance among the gradient-based filters.

Experiment – 8

Smoothing Filters

AIM: To apply and analyze mean, median, and convolution-based smoothing filters on a digital image to reduce noise and enhance quality.

Theory

1. Image Smoothing (Filtering)

Image smoothing is a technique used to reduce noise and variations in pixel intensity in order to produce a clearer and less distorted image. It helps in improving image quality by minimizing unwanted fluctuations while preserving important structures.

2. Median Filter

- A non-linear filter that replaces each pixel value with the median of its neighboring pixel values.
- Particularly effective for removing salt-and-pepper noise while preserving edges.

The median filtering operation is given by:

$$g(x,y) = \text{median}\{f(s,t)\}, \text{ for } (s,t) \in W$$

where:

- $f(s,t)$ is the original image
- $g(x,y)$ is the filtered image
- W is the neighborhood window around pixel (x,y)

3. Mean Filter (Average Filter)

- A linear filter that replaces each pixel with the average value of its neighboring pixels.
- It smooths the image but may cause blurring of edges.

The mean filtering operation is defined as:

$$g(x,y) = 1/N \sum_{(s,t) \in W} f(s,t)$$

where: N is the total number of pixels in the window W

4. Convolution Filtering

Convolution is a mathematical operation that combines an image with a kernel (filter matrix).

It is widely used for implementing:

- Averaging filters
- Sharpening filters
- Gaussian filters

In convolution, the kernel slides over the image, and the weighted sum of neighboring pixels is computed to generate the new pixel value.

Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_color = cv2.imread('./content/tiger.jpg')
img_color = cv2.cvtColor(img_color, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img_color, cv2.COLOR_RGB2GRAY)
noise_img = img_gray.copy()
prob = 0.02
# Salt noise
salt = np.random.rand(*img_gray.shape) < prob
noise_img[salt] = 255
# Pepper noise
pepper = np.random.rand(*img_gray.shape) < prob
noise_img[pepper] = 0
median_3 = cv2.medianBlur(noise_img, 3)
median_9 = cv2.medianBlur(noise_img, 9)
mean_3 = cv2.blur(img_gray, (3,3))
mean_10 = cv2.blur(img_gray, (10,10))
kernel_3 = np.ones((3,3), np.float32) / 9
kernel_10 = np.ones((10,10), np.float32) / 100
filtered_3 = cv2.filter2D(img_gray, -1, kernel_3)
filtered_10 = cv2.filter2D(img_gray, -1, kernel_10)

# ----- FIRST OUTPUT (Median Section) -----
plt.figure(figsize=(10,10))
plt.subplot(3,2,1)
plt.imshow(img_color)
plt.title("Original Image")
plt.axis('on')
plt.subplot(3,2,2)
plt.imshow(img_gray, cmap='gray')
plt.title("Gray Image")
plt.axis('on')
plt.subplot(3,2,3)
plt.imshow(noise_img, cmap='gray')
plt.title("Noise Added Image")
plt.axis('on')
plt.subplot(3,2,4)
plt.imshow(median_3, cmap='gray')
plt.title("3x3 Median Filter")
plt.axis('on')
plt.subplot(3,2,5)
plt.imshow(median_9, cmap='gray')
plt.title("9x9 Median Filter")
plt.axis('on')
plt.tight_layout()
plt.show()

# ----- SECOND OUTPUT (Mean Section) -----
plt.figure(figsize=(10,10))
plt.subplot(2,2,1)
```



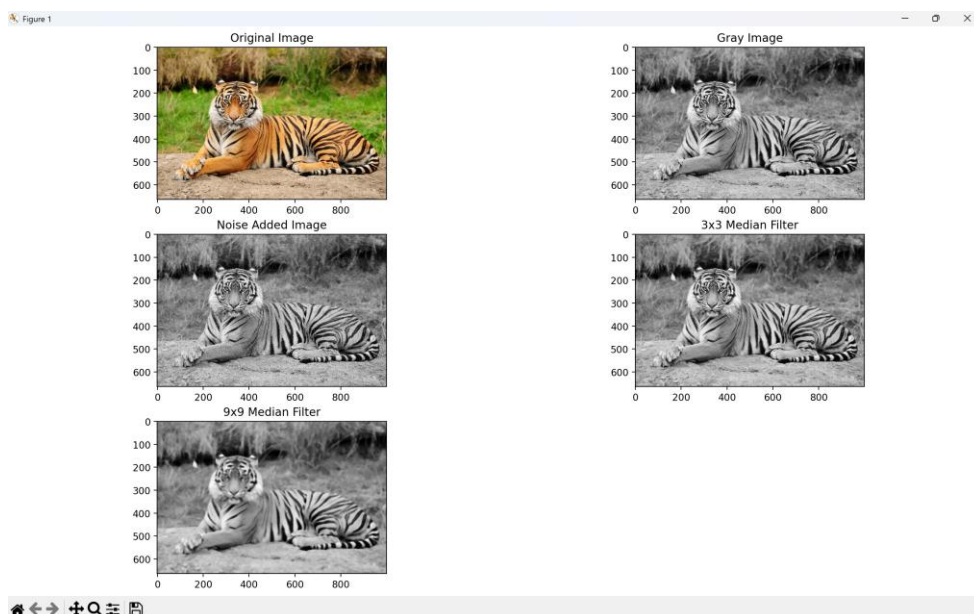
```

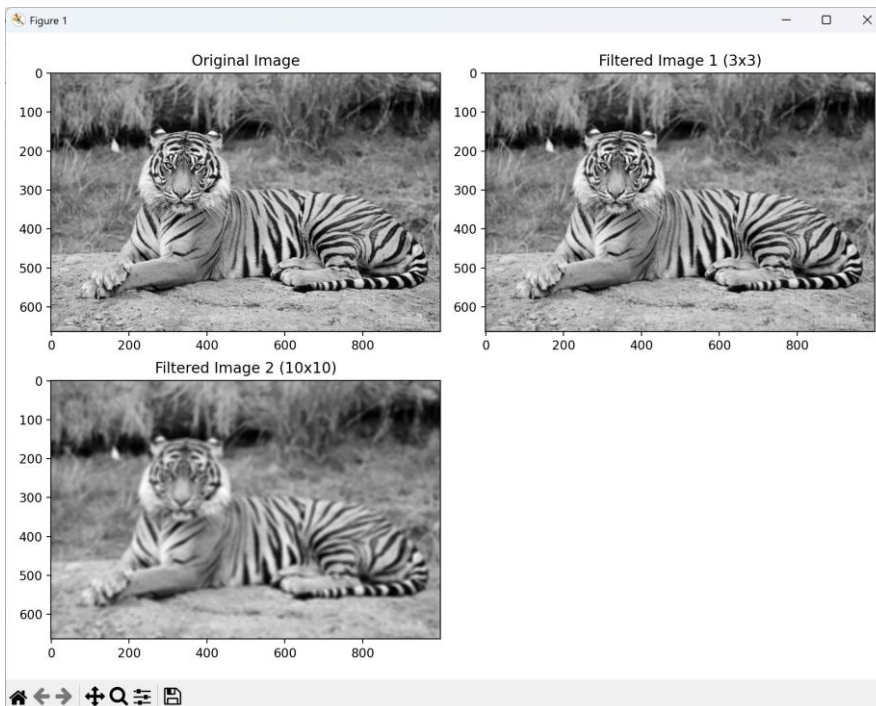
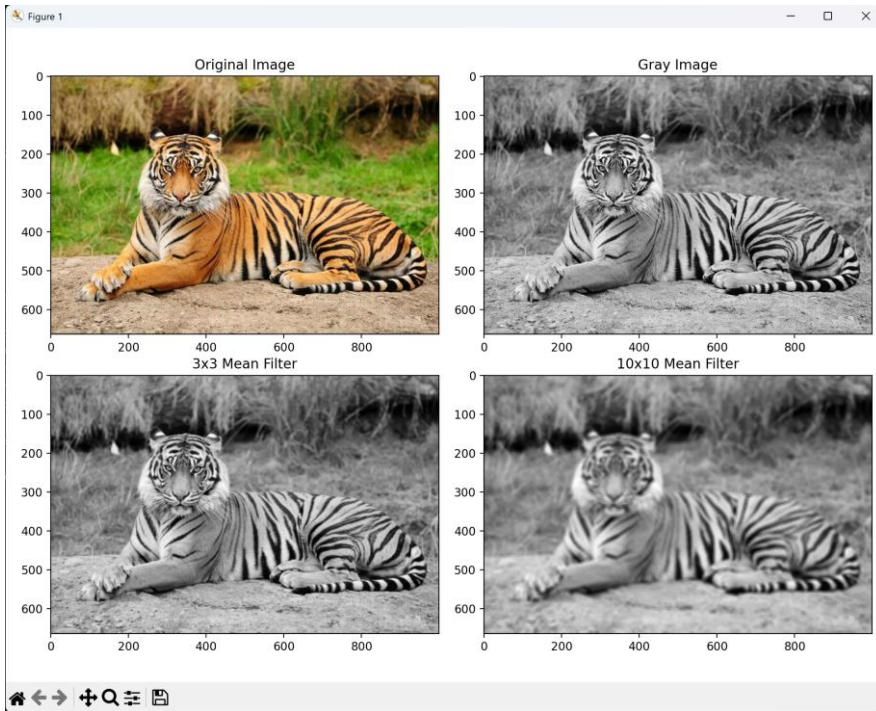
plt.imshow(img_color)
plt.title("Original Image")
plt.axis('on')
plt.subplot(2,2,2)
plt.imshow(img_gray, cmap='gray')
plt.title("Gray Image")
plt.axis('on')
plt.subplot(2,2,3)
plt.imshow(mean_3, cmap='gray')
plt.title("3x3 Mean Filter")
plt.axis('on')
plt.subplot(2,2,4)
plt.imshow(mean_10, cmap='gray')
plt.title("10x10 Mean Filter")
plt.axis('on')
plt.tight_layout()
plt.show()

# ----- THIRD OUTPUT (Convolution Section) -----
plt.figure(figsize=(10,10))
plt.subplot(2,2,1)
plt.imshow(img_gray, cmap='gray')
plt.title("Original Image")
plt.axis('on')
plt.subplot(2,2,2)
plt.imshow(filtered_3, cmap='gray')
plt.title("Filtered Image 1 (3x3)")
plt.axis('on')
plt.subplot(2,2,3)
plt.imshow(filtered_10, cmap='gray')
plt.title("Filtered Image 2 (10x10)")
plt.axis('on')
plt.tight_layout()
plt.show()

```

Output





Results and Discussion

The original RGB image was converted into grayscale and salt-and-pepper noise was added to simulate image degradation. The noisy image showed random white and black pixels, reducing visual clarity. After applying the **3×3 median filter**, most of the salt-and-pepper noise was removed while preserving important edges of the tree. The **9×9 median filter** further reduced noise but caused slight smoothing of fine details. This demonstrates that the median filter is highly effective for removing impulse noise while maintaining edge information. In the case of the **mean filter (3×3 and 10×10)**, the image appeared smoother; however, edges became blurred. The 10×10 mean filter produced stronger smoothing but resulted in noticeable loss of detail, confirming that larger kernels increase blurring. The **convolution-based filtering** using averaging kernels produced results similar to the mean filter, as it performs weighted averaging. Increasing kernel size increased smoothness but reduced sharpness.

Experiment – 10

Canny Edge Detection

AIM: To implement the Canny edge detection algorithm for detecting edges in an image.

Theory

Edge detection is a fundamental operation in Digital Image Processing used to identify boundaries of objects within an image. The Canny Edge Detection algorithm is one of the most effective and widely used methods for detecting edges with high accuracy and low error rate.

The Canny algorithm is a **multi-stage process** designed to detect a wide range of edges while minimizing noise and false detections. It provides good detection, good localization, and a single response to an edge.

Steps Involved in Canny Edge Detection

1. Noise Reduction

The image is first smoothed using a Gaussian filter to remove noise. This is necessary because noise can lead to false edge detection.

2. Gradient Calculation

The intensity gradient of the image is calculated using operators such as Sobel. This step finds the magnitude and direction of edges.

3. Non-Maximum Suppression

This step thins the edges by keeping only the local maxima (strongest edges) and suppressing weaker ones.

4. Double Thresholding

Two threshold values (low and high) are used:

- Strong edges: above high threshold
- Weak edges: between low and high
- Non-edges: below low threshold

5. Edge Tracking by Hysteresis

Weak edges connected to strong edges are preserved, while others are discarded. This ensures continuity of edges.

Code

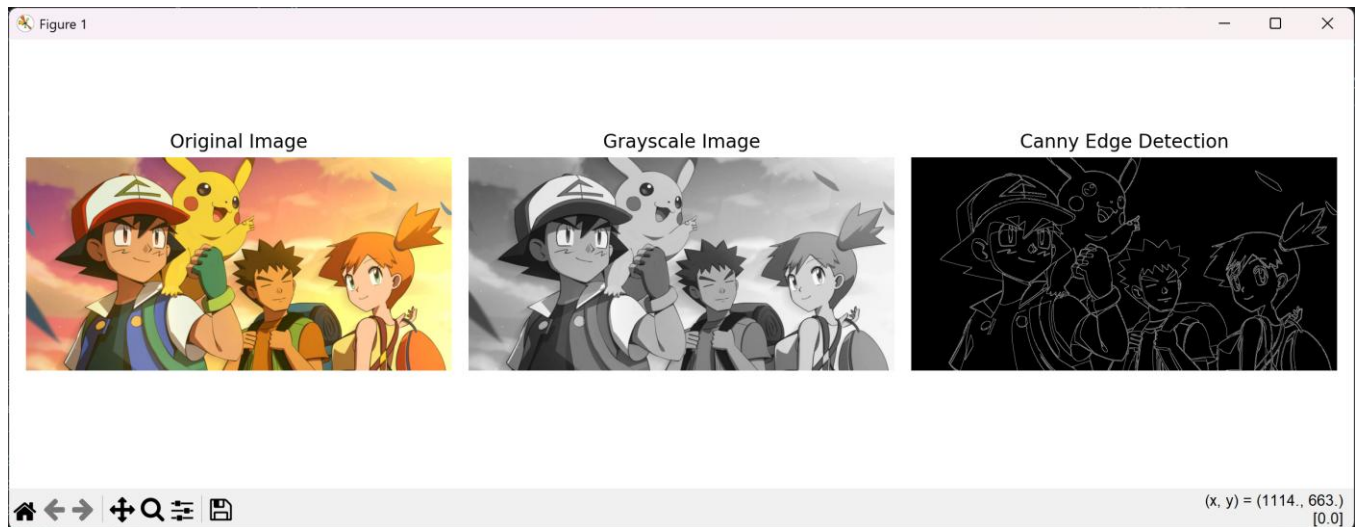
```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('./content/pokemon.jpeg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)

plt.figure(figsize=(12,4))
plt.subplot(1,3,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,3,2)
plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis('off')
```

```
plt.subplot(1,3,3)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Output



Results and Discussion

The input image was successfully converted to grayscale, reducing color information while preserving structural details. The Canny edge detection algorithm identified the major boundaries of the butterfly wings, flower petals, and antennae. The detected edges appear thin and well-localized, highlighting significant intensity transitions in the image.

The Canny algorithm effectively detected meaningful edges while suppressing non-significant variations. It produced sharper and more continuous edge boundaries compared to simple gradient operators. However, slight noise is visible in darker background regions, indicating sensitivity to threshold selection. Overall, the method provided accurate and clear edge detection for the given image.